Common Characters

AlgoExpert · Easy · Strings

The problem

- Return the unique characters that appear in every string.

Intuition

- A character is common if it appears in every string, so count in how many strings each unique character appears.

Brute force vs optimal

- Brute force: for each char of word 1, scan every other word
- Optimal: count distinct chars per word → O(total chars)
- Use a set per word so repeats count once

Approach

- 1. For each string, take its set of characters
- 2. Count the number of strings containing each one
- 3. Keep characters whose count equals the number of strings

Worked example

`word: abc`

char	#strings
a	1
b	1
c	1

After 'abc': counts updated

Worked example

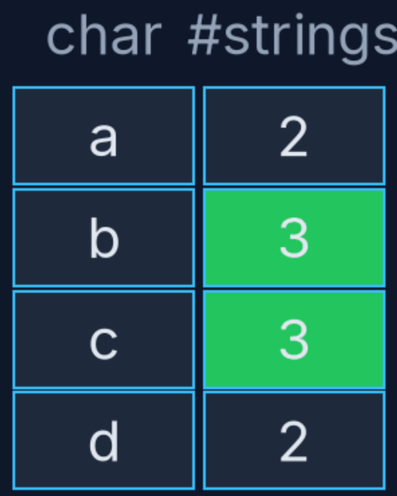
```
word: bcd
```

char	#strings
a	1
b	2
c	2
d	1

After 'bcd': counts updated

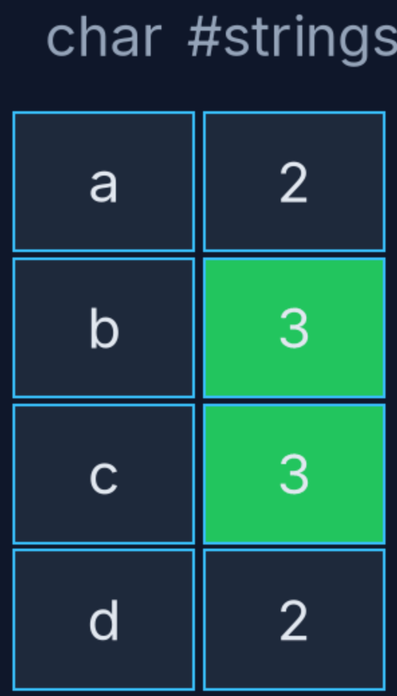
Worked example

`word: cbad`



After 'cbad': counts updated

Worked example



Answer: b, c

Java solution

```java
public class CommonCharacters {
    public static String[] commonCharacters(String[] strings) {
        Set<Character> common = null;
        for (String s : strings) {
            Set<Character> cur = new HashSet<>();
            for (char c : s.toCharArray()) cur.add(c);
            if (common == null) common = cur; else common.retainAll(cur);
        }
        List<String> out = new ArrayList<>();
        for (char c : new TreeSet<>(common)) out.add(String.valueOf(c));
        return out.toArray(new String[0]);
    }
}
```

Complexity

- Time: O(n*m)
- Space: O(c) distinct chars

Edge cases & pitfalls

- Count each char once per word (use a set)
- Single word → all its unique chars
- Empty string → no common chars

Interview tips & follow-ups

- Optimise: start from the shortest word's chars
- Follow-up: include multiplicity (min counts)

Recap

- A character is common if it appears in every string, so count in how many strings each unique character appears.
- Complexity: O(n*m)